

Object Oriented Programming

Lab Task 3



Submitted by:

Maham Shahzad

SP26-BAI-027

Submitted to:

Sir Shahid Bhatti

Department of Computer Science
COMSATS University Islamabad, Lahore Campus
24 September 2026

Activity 1

Methods & Method Overloading

Cart.java

```
public class Cart {
    private double total;

    public void addItem(double price) {
        if (price > 0) total += price;
    }

    // Task 1.1 – overloaded version that takes a quantity too
    public void addItem(double price, int quantity) {
        if (price > 0 && quantity > 0) {
            total += price * quantity;
        }
    }

    public double getTotal() {
        return total;
    }
}
```

Concept check (tied to Task 1)

Does a Java method signature include the return type? No — a signature is just the method name plus the number, types, and order of its parameters. The return type has nothing to do with it, which is exactly why the two addItem methods above are valid overloads (they differ in parameter list) while the Task 4 attempt below is not (same parameter list, only the return type differs).

Which member stores the cart state, and which members form its public API? The single private field total is the only thing holding state — nothing outside the class can see or touch it directly. The public API is the two addItem() overloads (the only ways to change total, both with validation) and getTotal() (the only way to read it, and read-only at that).

Demo.java

```
public class Demo {
    public static void main(String[] args) {
        Cart cart = new Cart();

        // Task 2: one notebook (Rs 120) + three pens at Rs 50 each
        cart.addItem(120);
        cart.addItem(50, 3);
        System.out.println("Total after valid purchases: " + cart.getTotal());

        // Task 3: invalid calls should not touch the total at all
        double before = cart.getTotal();
        cart.addItem(-10);
        cart.addItem(10, 0);
        System.out.println("Total after invalid calls: " + cart.getTotal());
        System.out.println("Total unchanged? " + (before == cart.getTotal()));

        // Task 4 – see the comment block below the class for the overload attempt
    }
}
```

Task 3 what happens with addItem(-10) and addItem(10, 0)

Neither call changes the total. addItem(-10) fails the price > 0 check in the single-argument version, and addItem(10, 0) fails the quantity > 0 check in the overloaded version. Both branches just fall through and do nothing, which is exactly the point of validating inside the method instead of trusting whatever the caller sends.

Task 4 trying to overload by return type only

I added this on purpose to see the compiler reject it, then commented it back out:

```
// Attempted invalid overload (return type changed, parameters identical):
// public int addItem(double price, int quantity) {
//     total += price * quantity;
//     return (int) total;
// }

// javac output:
// Cart.java:16: error: method addItem(double,int) is already defined in class Cart
//     public int addItem(double price, int quantity) {
//           ^
// 1 error
```

This confirms that Java overload resolution is based on the parameter list, not the return type — two methods with the same name and the same parameter types are the same signature no matter what they return, so the compiler just sees it as a duplicate declaration.

OUTPUT.txt

```
Total after valid purchases: 270.0
Total after invalid calls: 270.0
Total unchanged? true
```

README.txt

getTotal() is the only public method that reveals anything about the cart's state, and even that is read-only — you can't set the total from outside, you can only add to it through addItem(), and only when the input actually makes sense. That's the whole encapsulation idea in miniature.

Activity 2

What Does Java Actually Pass to a Method?

LabProgress.java (given, unchanged)

```
public class LabProgress {
    private int completedLabs = 1;
    public void completeOne() { completedLabs++; }
    public int getCompletedLabs() { return completedLabs; }
}
```

Demo.java

```
public class Demo {
    public static void main(String[] args) {
        int n = 4;
        LabProgress progress = new LabProgress();

        System.out.println("n before: " + n);
        System.out.println("completedLabs before: " + progress.getCompletedLabs());

        increaseNumber(n);
        System.out.println("n after increaseNumber: " + n);

        updateObject(progress);
        System.out.println("completedLabs after updateObject: " +
progress.getCompletedLabs());

        replaceLocal(progress);
        System.out.println("completedLabs after replaceLocal: " +
progress.getCompletedLabs());
    }

    static void increaseNumber(int n) { n += 3; }

    static void updateObject(LabProgress p) { p.completeOne(); }

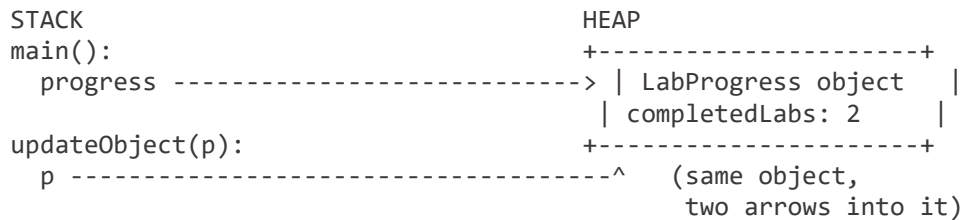
    static void replaceLocal(LabProgress p) {
        p = new LabProgress();
        p.completeOne();
    }
}
```

Predicted vs. actual output

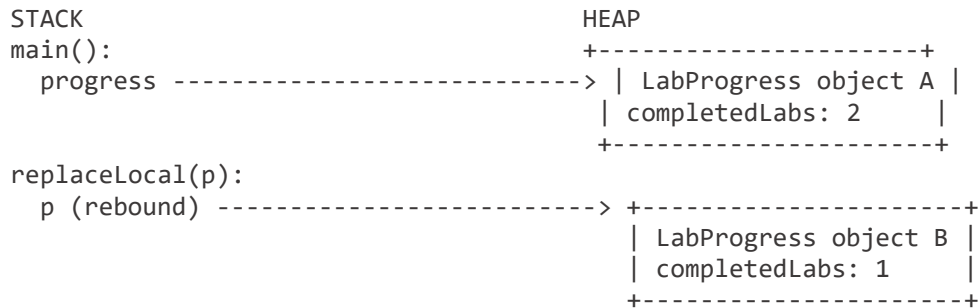
Checkpoint	Predicted	Actual
n, initially / after increaseNumber	4 / 4	4 / 4
progress, initially	1 completed lab	1 completed lab
After updateObject(progress)	2 completed labs	2 completed labs
After replaceLocal(progress)	still 2 completed labs	still 2 completed labs

Task 3 — stack/heap sketches

First sketch, during updateObject(progress) — the stack has the caller's variable progress and the parameter p sitting side by side, and both of them are just arrows pointing at the same LabProgress object on the heap:



Second sketch, after `p = new LabProgress();` runs inside `replaceLocal` — `p` gets rebound to point at a brand-new object, but `progress` in `main()` was never touched and still points at the original one:



Task 4 why `updateObject` changes the shared object but `replaceLocal` doesn't

`updateObject` never touches the reference itself — it just follows the copied reference to the one shared object and calls `completeOne()` on it, so the change happens on the object both variables point to. `replaceLocal` is different: the line `p = new LabProgress();` reassigns the local variable `p` to point somewhere else entirely, but `p` is only a copy of the reference that was passed in. Rebinding a copy doesn't rebind the original. `progress` in `main()` is still pointing at object A the whole time, completely unaffected by whatever `p` does afterward.

Activity 3

StudentProfile: Controlling Valid State

StudentProfile.java

```
public class StudentProfile {
    private String studentId;
    private double gpa;
    private int completedCredits;

    public boolean registerId(String id) {
        if (studentId == null && id != null && !id.trim().isEmpty()) {
            studentId = id;
            return true;
        }
        return false;
    }

    public boolean setGpa(double value) {
        if (value >= 0.0 && value <= 4.0) {
            gpa = value;
            return true;
        }
        return false;
    }

    public boolean addCredits(int amount) {
        if (amount > 0) {
            completedCredits += amount;
            return true;
        }
        return false;
    }

    public String getStudentId() { return studentId; }
    public double getGpa() { return gpa; }
    public int getCompletedCredits() { return completedCredits; }

    public String summary() {
        return studentId + " | GPA: " + gpa + " | Credits: " + completedCredits;
    }
}
```

Demo.java

```
public class Demo {
    public static void main(String[] args) {
        StudentProfile student = new StudentProfile();

        System.out.println(student.registerId("SP26-BCS-047")); // true
        System.out.println(student.registerId("NEW-ID"));        // false, already set

        System.out.println(student.setGpa(3.40)); // true
        System.out.println(student.setGpa(4.50)); // false, out of range

        System.out.println(student.addCredits(15)); // true
    }
}
```

```

        System.out.println(student.addCredits(-2)); // false
        System.out.println(student.addCredits(3)); // true, credits -> 18

        System.out.println(student.summary());
    }
}

```

Required test sequence — accepted / rejected

Action	Expected	What I observed
registerId("SP26-BCS-047")	true; ID stored	true — matches
registerId("NEW-ID")	false; original ID unchanged	false — matches
setGpa(3.40), then setGpa(4.50)	true, then false; GPA stays 3.40	true, false — matches
addCredits(15), then addCredits(-2)	true, then false; credits stay 15	true, false — matches
Add 3 more credits	Credits become 18	18 — matches

OUTPUT.txt

```

true
false
true
false
true
false
true
SP26-BCS-047 | GPA: 3.4 | Credits: 18

```

Concept check

A getter can be public while the field behind it is private because the getter only ever hands out a copy of the value (or a read-only view) — it never lets the caller reach in and overwrite the field. There's no reason to give every field a matching setter, because a blind setXxx() re-opens exactly the hole encapsulation was supposed to close: it lets a caller assign a GPA of 8.0 or a negative credit count just by calling a plain assignment method instead of going through a validated one. Fields that shouldn't change after creation (like studentId) shouldn't have a general setter at all.

Activity 4

DigitalWallet: Designing a Small API

DigitalWallet.java

```
public class DigitalWallet {
    private double balance;

    public boolean deposit(double amount) {
        if (amount > 0) {
            balance += amount;
            return true;
        }
        return false;
    }

    public boolean spend(double amount) {
        if (amount > 0 && amount <= balance) {
            balance -= amount;
            return true;
        }
        return false;
    }

    public double getBalance() {
        return balance;
    }

    public boolean canAfford(double amount) {
        return amount > 0 && amount <= balance;
    }
}
```

Demo.java

```
public class Demo {
    public static void main(String[] args) {
        DigitalWallet wallet1 = new DigitalWallet();
        DigitalWallet wallet2 = new DigitalWallet();

        run("deposit(500)", wallet1.deposit(500), wallet1);
        run("deposit(-10)", wallet1.deposit(-10), wallet1);
        run("spend(200)", wallet1.spend(200), wallet1);
        run("spend(1000)", wallet1.spend(1000), wallet1);
        run("canAfford(301)", wallet1.canAfford(301), wallet1);
        run("spend(300)", wallet1.spend(300), wallet1);

        System.out.println("wallet2 balance (never touched): " + wallet2.getBalance());

        // Task 4: try to bypass the API directly
        // wallet1.balance = -500;
        // javac error:
        // Demo.java:20: error: balance has private access in DigitalWallet
    }

    static void run(String label, boolean result, DigitalWallet w) {
```

```

        System.out.println(label + " -> " + result + ", balance = " + w.getBalance());
    }
}

```

Test plan for wallet 1

Call	Return	Expected balance
deposit(500)	true	500
deposit(-10)	false	500
spend(200)	true	300
spend(1000)	false	300
canAfford(301)	false	300
spend(300)	true	0

OUTPUT.txt

```

deposit(500) -> true, balance = 500.0
deposit(-10) -> false, balance = 500.0
spend(200) -> true, balance = 300.0
spend(1000) -> false, balance = 300.0
canAfford(301) -> false, balance = 300.0
spend(300) -> true, balance = 0.0
wallet2 balance (never touched): 0.0

```

Concept check

deposit() only ever moves the balance in one controlled direction and rejects non-positive amounts, whereas a public setBalance(double) would let a caller set the balance to literally anything — including negative numbers or some arbitrary value that has nothing to do with an actual transaction. Because balance is private and there's no setter for it, a line like wallet.balance = -500; simply doesn't compile anymore — that invalid operation is no longer reachable from outside the class.

Activity 5

Package Boundaries & Visibility

Folder layout

```
A5_PackageVisibility/  
  model/  
    AccessBox.java  
    SamePackageCheck.java  
  app/  
    OutsideCheck.java
```

model/AccessBox.java (given)

```
package model;  
  
public class AccessBox {  
    public int open = 1;  
    protected int family = 2;  
    int packageOnly = 3;  
    private int secret = 4;  
  
    public void printInside() {  
        System.out.println(open + " " + family + " " + packageOnly + " " + secret);  
    }  
}
```

model/SamePackageCheck.java

```
package model;  
  
public class SamePackageCheck {  
    public static void main(String[] args) {  
        AccessBox box = new AccessBox();  
        System.out.println(box.open);  
        System.out.println(box.family);  
        System.out.println(box.packageOnly);  
        // System.out.println(box.secret); // private – not visible even in same package  
        box.printInside();  
    }  
}
```

app/OutsideCheck.java

```
package app;  
  
import model.AccessBox;  
  
public class OutsideCheck {  
    public static void main(String[] args) {  
        AccessBox box = new AccessBox();  
        box.printInside();  
        System.out.println(box.open);  
        // System.out.println(box.family); // protected, different package, no  
inheritance // System.out.println(box.packageOnly); // default access, different package  
        // System.out.println(box.secret); // private, never visible outside the  
class
```

```
}  
}
```

Compile & run

```
mkdir out  
javac -d out model\AccessBox.java model\SamePackageCheck.java app\OutsideCheck.java  
java -cp out model.SamePackageCheck  
java -cp out app.OutsideCheck
```

Prediction and error experiment (NOTES.txt)

Uncommenting `box.secret` in either file gives the same kind of error no matter where it's tried, because private members belong strictly to the declaring class:

```
OutsideCheck.java:10: error: family has protected access in model.AccessBox  
    System.out.println(box.family);  
                        ^  
1 error  
  
SamePackageCheck.java:9: error: secret has private access in model.AccessBox  
    System.out.println(box.secret);  
                        ^  
1 error
```

All the commented-out lines were restored before the folder was considered done, as the instructions asked.

Access summary

Member	Inside class	Same package	Different package*
public open	Yes	Yes	Yes
protected family	Yes	Yes	No
packageOnly (default)	Yes	Yes	No
private secret	Yes	No	No

* *Different-package client here is a plain, unrelated class — not a subclass. Cross-package protected access through inheritance is a separate case covered later once inheritance is introduced.*

Concept check

They all have to agree because the compiler and the JVM use them together to locate a class: the package declaration at the top of the file has to match the folder the file physically lives in relative to the source root, `javac -d out` places the compiled `.class` file inside a matching `out/model` or `out/app` folder, and `java -cp out model.SamePackageCheck` then looks for `model/SamePackageCheck.class` under `out`. If any one of those is off, the class simply won't be found. The visibility boundary itself (public/protected/default/private) is enforced at compile time — the compiler rejects the illegal access before a `.class` file is even produced, it's not something checked while the program is running.

Activity 6

Integration Challenge: LibraryBook

LibraryBook.java

```
public class LibraryBook {
    private int totalCopies;
    private int borrowedCopies;
    private boolean configured;

    public boolean configure(int total) {
        if (!configured && total > 0) {
            totalCopies = total;
            configured = true;
            return true;
        }
        return false;
    }

    public boolean borrow() {
        return borrow(1);
    }

    public boolean borrow(int n) {
        if (n <= 0 || n > getAvailable()) {
            return false;
        }
        borrowedCopies += n;
        return true;
    }

    public boolean returnCopies(int n) {
        if (n <= 0 || n > borrowedCopies) {
            return false;
        }
        borrowedCopies -= n;
        return true;
    }

    public int getAvailable() {
        return totalCopies - borrowedCopies;
    }

    public int getBorrowed() {
        return borrowedCopies;
    }
}
```

Demo.java

```
public class Demo {
    public static void main(String[] args) {
        LibraryBook book1 = new LibraryBook();
        LibraryBook book2 = new LibraryBook();

        book1.configure(3);
    }
}
```

```

        book2.configure(1);
        System.out.println("book1 available: " + book1.getAvailable() + ", book2 available: " + book2.getAvailable());

        run("borrow(2)", book1.borrow(2), book1);
        run("borrow(2) again", book1.borrow(2), book1);
        run("returnCopies(1)", book1.returnCopies(1), book1);
        run("returnCopies(2)", book1.returnCopies(2), book1);
        run("borrow()", book1.borrow(), book1);

        // Task 4: reconfiguring after setup must fail and must not disturb existing state
        System.out.println("configure(10) again -> " + book1.configure(10));
        System.out.println("book1 borrowed/available unchanged: " + book1.getBorrowed() + "/" + book1.getAvailable());
    }

    static void run(String label, boolean result, LibraryBook b) {
        System.out.println(label + " -> " + result + ", borrowed/available = " + b.getBorrowed() + "/" + b.getAvailable());
    }
}

```

Required checks for book 1

Operation	Result	Borrowed / available
configure(3)	true	0 / 3
borrow(2)	true	2 / 1
borrow(2)	false	2 / 1
returnCopies(1)	true	1 / 2
returnCopies(2)	false	1 / 2
borrow()	true	2 / 1

OUTPUT.txt

```

book1 available: 3, book2 available: 1
borrow(2) -> true, borrowed/available = 2/1
borrow(2) again -> false, borrowed/available = 2/1
returnCopies(1) -> true, borrowed/available = 1/2
returnCopies(2) -> false, borrowed/available = 1/2
borrow() -> true, borrowed/available = 2/1
configure(10) again -> false
book1 borrowed/available unchanged: 2/1

```

Concept check

In plain words, the invariant is: a book can never have more copies out on loan than it actually owns, and it can never have a negative number borrowed either — `borrowedCopies` always sits somewhere between 0 and `totalCopies`. A public `setBorrowedCopies(int)` would break that guarantee immediately, because it would let calling code jump straight to any number — including something above `totalCopies` or below zero — without going through `borrow()` or `returnCopies()`, which are the only two places that actually check the invariant before changing the count.

Final Checkpoint

1. Which parameter lists make the two `Cart.addItem` methods valid overloads? Why is a changed return type alone insufficient?

`addItem(double)` and `addItem(double, int)` are valid overloads because they differ in the number of parameters — one takes a single price, the other takes a price and a quantity. That's what the compiler actually looks at when it resolves which method to call: the parameter list. The return type isn't part of a method's signature at all, so two methods with identical parameters but different return types still count as the exact same signature, and the compiler just reports a duplicate method rather than picking between them.

2. Why does `increaseNumber(n)` not modify the caller's integer in Activity 2?

`n` is an `int`, a primitive, so Java copies its value into the parameter when the method is called. Inside `increaseNumber`, the local copy goes from 4 to 7, but that copy lives entirely inside the method's own stack frame and disappears once the method returns. The `n` back in `main()` was never touched — it was only ever the source of a value that got copied, not something the method had a handle on.

3. Why can `updateObject(progress)` change the original object even though Java is pass-by-value?

Java is pass-by-value even for objects — what gets copied is the reference (the address), not the object itself. So `updateObject` gets its own copy of the reference, but that copy still points at the exact same `LabProgress` object on the heap as `progress` does in `main()`. Calling `p.completeOne()` follows that reference to the shared object and changes its field, and since there's only one object involved, the change is visible everywhere, including back in `main()`.

4. Why does rebinding `p` inside `replaceLocal(p)` not redirect the caller's reference?

Because `p` is a local copy of the reference, rebinding it with `p = new LabProgress()` only changes what that local copy points to — it doesn't reach back and change what `progress` in `main()` points to. The two variables were pointing at the same object before the reassignment, but they were never the same variable. After `p` is rebound, `progress` still points at the original object; `p` just wandered off on its own.

5. Name two `StudentProfile` invariants and identify the method responsible for maintaining each one.

First invariant: `gpa` always stays between 0.0 and 4.0, and that's enforced entirely inside `setGpa()`, which rejects anything outside that range before it ever gets assigned. Second invariant: `studentId`, once set, can never be silently replaced — `registerId()` only succeeds the first time (when `studentId` is still null), so a second call with a different ID just returns false and leaves the original untouched.

6. Why does `DigitalWallet` provide `deposit()/spend()` but no public `setBalance()`?

`deposit()` and `spend()` describe actual real-world actions and each one enforces its own rule (`deposit` rejects non-positive amounts, `spend` also checks there's enough balance to cover it), so every change to balance has to go through a check first. A public `setBalance()` would skip all of that — it would let a caller assign any number at all, including a negative balance or a balance that has no relationship to any real transaction, which is exactly the kind of shortcut encapsulation is meant to close off.

7. Which `AccessBox` members are readable by an unrelated class in another package? What changes for a client in the same package?

From an unrelated class in a different package (like `app.OutsideCheck`), only the public field `open` is directly reachable — `family`, `packageOnly`, and `secret` are all off-limits from there. A client in the same package as `AccessBox` (`model.SamePackageCheck`) gains access to `open`, `family`, and `packageOnly` as well, since `protected` and `default` (`package-private`) access both extend to any class sharing the same package. `secret`

stays private in every case — it's never visible outside AccessBox itself, no matter which package the caller is in.

8. Write the LibraryBook invariant and identify two rejected operations in the test sequence.

Invariant: borrowedCopies must always stay between 0 and totalCopies (inclusive) — a book can never lend out more copies than it has, and the borrowed count can never drop below zero. Two rejected operations from the test sequence: the second borrow(2) call was rejected because only 1 copy was available at that point (2 were already out of 3), and returnCopies(2) was rejected because only 1 copy was actually out on loan at that point, so returning 2 would have pushed borrowedCopies negative.

Exit ticket:

A carefully designed public API keeps an object valid because every path that could change its state is forced through a method that checks the input first — there's no back door where client code can just overwrite a field directly. Even if the calling code makes a mistake (a negative amount, an empty ID, borrowing more copies than exist), the object itself refuses the bad change and reports back that it failed, instead of silently ending up in a broken state.